

GitHub Copilot Workshop

From AI Fundamentals to Hands-on Refactoring

4-Hour Workshop

Agenda 📅

Part 1: Foundations (1h 15min)



Introduction to LLMs



Introduction to GitHub Copilot



Ask / Edit / Agent Modes

Break (15min)



Part 2: Mastering Copilot (45min)



Context is King



Fine-tuning GitHub Copilot

Break (15min)



Part 3: Hands-on Lab (1h 15min)



SupermarketReceipt Refactoring Kata

Part 1.1



Introduction to LLMs

What is a Large Language Model?

A **Large Language Model (LLM)** is an AI system trained to understand and generate human-like text.

Key Insight: LLMs don't "understand" like humans do. They predict the most likely next token based on patterns learned from training data.

Think of it as **sophisticated autocomplete** that has read most of the internet.

How LLMs Work: The Basics

Input Text → Tokenization → Neural Network → Probability Distribution → Output Token
↑
_____ Repeat until done _____

1. **Tokenization:** Text split into tokens (words, subwords, characters)
2. **Embedding:** Tokens converted to numerical vectors
3. **Transformer:** Neural network processes context & relationships
4. **Prediction:** Model outputs probability for each possible next token
5. **Generation:** Sample from probabilities, repeat

Training LLMs: Three Phases

Phase	What Happens	Data Used
Pre-training	Learn language patterns	Massive text corpus (internet, books)
Fine-tuning	Specialize for tasks	Curated datasets for specific domains
RLHF	Align with human preferences	Human feedback on outputs

For Code: Models like Codex/GPT-4 were fine-tuned on millions of code repositories, learning syntax, patterns, and best practices across languages.

LLM Strengths

- ✓ **Pattern recognition** at massive scale
- ✓ **Code generation** from natural language
- ✓ **Language translation** (including programming languages!)
- ✓ **Summarization** and explanation
- ✓ **Boilerplate elimination** - write less repetitive code
- ✓ **Learning accelerator** - explore unfamiliar codebases faster
- ✓ **Multi-language fluency** - works across 40+ programming languages

LLM Limitations

- ✗ **Hallucinations** - confidently wrong answers
- ✗ **No true understanding** - pattern matching, not reasoning
- ✗ **Training cutoff** - knowledge stops at training date
- ✗ **Context window limits** - can only "see" limited text at once
- ✗ **Non-deterministic** - same prompt may give different results
- ✗ **Security blind spots** - may suggest vulnerable code
- ✗ **License unawareness** - doesn't track code licenses

The "Stochastic Parrot" Reality

What LLMs ARE

- Statistical pattern matchers
- Probability engines
- Text transformers
- Excellent at interpolation

What LLMs ARE NOT

- Databases of facts
- Search engines
- Reasoning engines
- Reliable for novel problems

Developer Mindset: Use Copilot as a **very fast junior developer** who knows a lot but makes mistakes. Always review!

Why This Matters for Developers

Understanding LLM limitations helps you:

1. **Write better prompts** - provide context the model needs
2. **Verify outputs** - know when to trust vs. double-check
3. **Use efficiently** - pick the right tool for the job
4. **Stay secure** - review generated code critically

Remember: GitHub Copilot is a tool to **augment** your skills, not replace your judgment.

Part 1.2



Introduction to GitHub Copilot

GitHub Copilot: A Brief History

Date	Milestone
June 2021	Technical preview launched (OpenAI Codex)
June 2022	General availability (\$10/month)
March 2023	Copilot Chat introduced (GPT-4 based)
November 2023	Copilot Enterprise announced
October 2024	Multi-model support - "Era of single model is over"
February 2025	Agent mode GA in VS Code

GitHub Copilot has evolved from autocomplete → chat assistant → **autonomous agent**

Multi-Model Architecture (2024+)

GitHub Copilot now supports **multiple AI models** you can choose:

Provider	Models	Best For
OpenAI	GPT-4o, GPT-4.1, o1-preview	General coding, speed
Anthropic	Claude 3.5 Sonnet, Claude Sonnet 4	Nuanced understanding, safety
Google	Gemini 1.5 Pro, Gemini 2.0 Flash	Long context, multimodal
xAI	Grok	Alternative perspective

You can **switch models mid-conversation** based on the task!

Where Does Copilot Work?

IDEs & Editors

-  VS Code (primary)
-  Visual Studio 2022+
-  JetBrains (Rider, IntelliJ, etc.)
-  Neovim / Vim
-  Eclipse
-  Xcode

Beyond the IDE

-  GitHub.com (issues, PRs)
-  GitHub Mobile
-  GitHub CLI (`gh copilot`)
-  Windows Terminal
-  GitHub Copilot Workspace

Pricing Tiers (2025)

Tier	Price	Key Features
Free	\$0	2,000 completions/month, 50 chat messages
Pro	\$10/month	Unlimited completions, 300 premium requests
Pro+	\$39/month	1,500 premium requests, all models
Business	\$19/user/month	Org policies, IP indemnity, data excluded
Enterprise	\$39/user/month	Knowledge bases, fine-tuning (beta)

 **Free for:** Students, educators, and open-source maintainers

Copilot Features Overview

Code Completion

- Inline suggestions as you type
- Multi-line completions
- Context-aware from open files

Copilot Chat

- Natural language conversations
- Code explanations
- Bug fixing assistance

Agent Capabilities

- Autonomous multi-step tasks
- Terminal command execution
- Self-correction loops

AI Security Best Practices

Working Safely with AI Tools

The Golden Rules of AI Security

Protect Your Organization

NEVER Share:

-  API keys or tokens
-  Passwords or credentials
-  Customer PII data
-  Proprietary algorithms
-  Sensitive business data

ALWAYS:

-  Use enterprise AI tools
-  Review generated code
-  Check data retention policies
-  Follow company guidelines
-  Report security concerns

Free vs Enterprise AI Tools

Know the Difference

✗ Free/Consumer AI:

- Data used for training
- No privacy guarantees
- Logs conversations
- Public model updates
- No audit trail

Risk: Code becomes public!

✓ GitHub Copilot Business / Enterprise:

- Zero data retention
- No training on your code
- SOC 2 compliant
- Audit logs available
- Enterprise controls

Safe: Your code stays yours!

Common Security Mistakes

What We've Seen Go Wrong

Real Incidents:

1. **API Key Exposure:** Developer asked ChatGPT to debug code with live AWS keys
2. **Customer Data Leak:** Pasted real customer database queries into free AI
3. **Algorithm Theft:** Proprietary trading logic ended up in public training data
4. **Compliance Violation:** GDPR data processed through non-compliant AI

Prevention: Think before you paste!

Safe AI Practices

Your Security Checklist

Before Using Any AI Tool:

- ☐ Is this tool approved by IT?
- ☐ Have I removed all secrets?
- ☐ Is the data anonymized?
- ☐ Do I understand retention?
- ☐ Am I following policy?

Pro Tip: Create test data sets for AI interactions

Handling Sensitive Code

When You Must Work with Secrets

Best Practices:

1. **Use placeholders:** Replace real values with `<API_KEY_HERE>`
2. **Environment variables:** Reference, don't embed
3. **Mock data:** Create realistic but fake examples
4. **Sanitize first:** Remove before sharing with AI

Example:

```
// DON'T: client.ApiKey = "sk-1234abcd...";  
// DO: client.ApiKey = Environment.GetVariable("API_KEY");
```

GitHub Copilot Security Features

Built-in Protections

Automatic Filtering:

- Blocks common secret patterns
- Prevents generating real API keys
- Filters personally identifiable information
- Excludes files in .gitignore

Your Controls:

- Disable for specific files
- Exclude repositories
- Review telemetry settings
- Configure organization policies

Part 1.3



Ask / Edit / Agent Modes

The Three Modes of Interaction

GitHub Copilot offers **three distinct interaction modes**, each with different levels of autonomy:

Mode	Autonomy	You Control	Copilot Does
Ask	Low	Everything	Answers questions
Edit	Medium	File scope	Proposes changes
Agent	High	Approval	Plans & executes

Choosing the right mode is key to efficient Copilot usage!

Ask Mode (Copilot Chat)

What it is: Conversational interface for questions and exploration

Access points (VS Code):

- Chat View: `Ctrl+Alt+I` (dedicated panel)
- Inline Chat: `Ctrl+I` (in editor)
- Quick Chat: `Ctrl+Shift+I` (floating)

Key characteristic:

No automatic code changes - you copy/paste or apply manually

Ask Mode: Slash Commands

Speed up common tasks with built-in commands:

Command	Purpose
<code>/explain</code>	Explain selected code
<code>/fix</code>	Suggest fixes for problems
<code>/tests</code>	Generate unit tests
<code>/doc</code>	Generate documentation
<code>/new</code>	Scaffold new project/file
<code>/clear</code>	Clear chat history

Example: Select a method → `/explain` → Get walkthrough

Ask Mode: Best Use Cases

✓ Learning & Understanding

- "Explain this LINQ query"
- "What design pattern is used here?"

✓ Exploration

- "What are different ways to implement caching?"
- "Compare async/await vs. Task.Run"

✓ Quick Answers

- "What's the C# syntax for pattern matching?"
- "How do I configure dependency injection?"

Edit Mode

What it is: Describe changes in natural language, get inline diffs

How it works:

1. You specify which files to include (working set)
2. Describe the change you want
3. Copilot proposes **diffs across multiple files**
4. You review and accept/reject per file

Key characteristic:

You control scope, Copilot implements

Edit Mode: How to Use

1. **Open Edit Mode:** Click mode selector or `Ctrl+Shift+I` → Select "Edit"
2. **Add files to working set:** Click `+` to add relevant files
3. **Describe your change:** "Add null checking to all public methods"
4. **Review changes:** See inline diffs with Accept/Reject buttons
5. **Apply or iterate:** Accept changes or refine your request

Pro tip: Be specific about scope. "In this file" vs "across the project"

Edit Mode: Best Use Cases

✓ Targeted Refactoring

- "Convert these callbacks to async/await"
- "Extract this logic into a separate method"

✓ Cross-file Changes

- "Add logging to all repository methods"
- "Update all DTOs to use records"

✓ Implementation Tasks

- "Implement the IDisposable pattern here"
- "Add input validation to this controller"

Agent Mode

What it is: Autonomous assistant that plans and executes multi-step tasks

Key capabilities:

-  **Discovers** relevant files automatically
-  **Plans** implementation steps
-  **Executes** terminal commands (with your approval)
-  **Self-corrects** when errors occur
-  **Creates/modifies** multiple files

Available in VS Code 1.99+ (GA February 2025)

Agent Mode: The Agentic Loop

Your Prompt



PLAN: Analyze task, find files



CODE: Generate/modify code



RUN: Execute terminal commands



CHECK: Verify results



If errors → Loop back to PLAN

← Your approval required



Solution (you review & accept)

Agent Mode: Example in Action

Prompt: "Add user authentication to this API"

Agent actions:

1. Scans codebase to understand structure
2. Creates plan: "I'll add JWT auth, create User model, update controllers"
3. Installs packages: `dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer`
4. Creates `User.cs`, `AuthController.cs`, `JwtService.cs`
5. Modifies `Program.cs` for auth middleware
6. Runs tests, fixes any failures
7. Presents complete solution for review

Agent Mode: Best Use Cases

✓ Open-ended tasks

- "Build a REST API for managing products"
- "Add a caching layer to the application"

✓ Project setup

- "Set up a new .NET Web API with clean architecture"
- "Configure CI/CD pipeline"

✓ Complex implementations

- "Implement the repository pattern with Unit of Work"
- "Add comprehensive error handling"

Choosing the Right Mode

Scenario	Mode
"How does this authentication work?"	Ask
"What's the best way to handle this?"	Ask
"Refactor these 3 files to use async/await"	Edit
"Add validation to this specific form"	Edit
"Build a complete user management feature"	Agent
"Set up the project from scratch"	Agent

Rule of thumb: Start with Ask to understand, use Edit for focused changes, use Agent for bigger tasks

Mode Comparison Summary

Aspect	Ask	Edit	Agent
Changes code	No	Yes (with approval)	Yes (with approval)
Finds files	Manual	Manual	Automatic
Runs commands	No	No	Yes
Multi-step	No	Limited	Yes
Self-corrects	No	No	Yes
Best for	Learning	Refactoring	Features

Demo: The Three Modes

Let's see each mode in action:

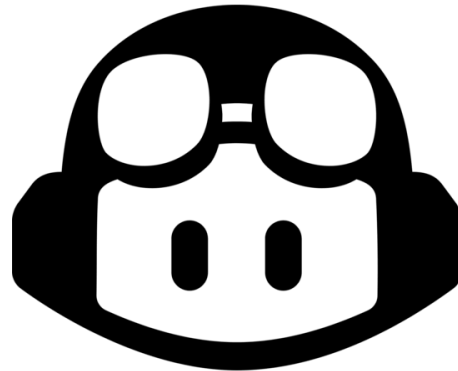
1. **Ask:** Explain the `handleOffers` method
2. **Edit:** Add null checks to a specific file
3. **Agent:** Add a new discount type



Break

15 minutes

Mastering Copilot



Part 2.1

Context is King

Why Context Matters

Poor Context

```
"Write a function to  
calculate discount"
```

Result: Generic, often wrong

Rich Context

```
"In our .NET 8 e-commerce app  
using the Money type from  
NodaMoney, write a method  
to calculate percentage  
discount with rounding"
```

Result: Specific, usable

More context = Better output. This is the #1 factor in Copilot effectiveness.

How Copilot Gathers Context

Automatic context sources:

-  Current file (highest priority)
-  Selected text
-  Open tabs in editor
-  File names and extensions
-  Language/framework detection

NOT automatically included:

- Git history
- Files in `.gitignore`
- Binary files
- Other projects

Reference Syntax: Files & Folders

Explicitly include context with references:

<code>#file:src/Models/Order.cs</code>	← Specific file
<code>#file:appsettings.json</code>	← Config file
<code>#folder:src/Services</code>	← Entire folder
<code>#folder:tests</code>	← Test folder

Example prompt:

Looking at `#file:src/Services/OrderService.cs`, refactor the `CalculateTotal` method to use LINQ

Reference Syntax: Symbols & Codebase

<code>#symbol:OrderProcessor</code>	← Class or function
<code>#symbol:HandleDiscount</code>	← Specific method
<code>#codebase</code>	← Search entire workspace
<code>#selection</code>	← Current selection

Example:

```
#codebase How is authentication implemented  
in this project?
```

Chat Participants: @workspace

@workspace - Makes Copilot an expert on your entire codebase

```
@workspace Where is the shopping cart logic?
```

```
@workspace What patterns does this project use  
for data access?
```

```
@workspace Find all usages of the Discount class
```

@workspace vs #codebase:

- **@workspace** = Single comprehensive search
- **#codebase** = Tool the agent can call multiple times

Chat Participants: Others

Participant	Purpose
@workspace	Query entire codebase
@terminal	Reference terminal context
@vscode	VS Code settings & commands
@github	GitHub-specific queries (PRs, issues)

Example:

@terminal What was the last error message?

@vscode How do I configure auto-save?

Custom Instructions File

Create `.github/copilot-instructions.md` in your repo root:

```
# Project Guidelines

## Tech Stack
- .NET 8 with C# 12
- Entity Framework Core 8
- xUnit with FluentAssertions

## Coding Standards
- Use PascalCase for public members
- Private fields prefixed with underscore
- Always use async/await for I/O operations
- Prefer records for DTOs
```

These instructions are **automatically included** in every Copilot interaction!

Example: copilot-instructions.md

```
# .NET Standards

## Architecture
- Follow Clean Architecture pattern
- Use CQRS for complex domains
- Repository pattern for data access

## Error Handling
- Use Result<T> pattern, not exceptions for expected failures
- Log all exceptions with correlation IDs

## Testing
- Minimum 80% code coverage
- Use builder pattern for test data
- One assertion concept per test
```


File-Specific Instructions

Create `.github/instructions/*.instructions.md` with `applyTo` patterns:

```
---
applyTo: "**/*.cs"
---
# C# Specific Guidelines
- Use file-scoped namespaces
- Prefer primary constructors where applicable
- Use collection expressions []
```

```
---
applyTo: "**/Tests/**"
---
# Test Guidelines
- Use Arrange-Act-Assert pattern
- Use descriptive test names: Should_DoX_When_Y
```

Context Best Practices

1. **Open relevant files** before asking questions
2. **Select specific code** when asking about it
3. **Use references** (`#file:` , `#folder:`) for precision
4. **Create instruction files** for project standards
5. **Start specific**, broaden if needed
6. **Check context indicator** in Copilot panel

Pro tip: If Copilot gives wrong answers, the problem is usually **missing context**, not the model.

Demo: Context in Action

Let's see the difference context makes:

1. **Without context:** Generic question
2. **With file reference:** Specific to our code
3. **With @workspace:** Project-wide understanding
4. **With instructions file:** Follows our standards

Part 2.2



Fine-tuning GitHub Copilot

Customization Options Overview

Feature	Availability	Purpose
Instructions file	All tiers	Project-specific guidelines
Prompt files	All tiers	Reusable task templates
Custom agents	Business+	Domain-specific experts
Extensions	All tiers	Third-party integrations
Knowledge bases	Enterprise	Organization docs
Fine-tuning	Enterprise (beta)	Model customization

Prompt Files (.prompt.md)

Create reusable prompts in `.github/prompts/`:

```
<!-- .github/prompts/code-review.prompt.md -->
---
mode: ask
---
Review this code for:
1. Potential bugs or edge cases
2. Performance issues
3. Security vulnerabilities
4. Adherence to SOLID principles

Provide specific suggestions with code examples.
```

Use with: `/code-review` in chat

Prompt File Examples

Test generation prompt:

```
<!-- .github/prompts/test.prompt.md -->
---
mode: ask
---
Generate xUnit tests for #selection using:
- FluentAssertions for assertions
- NSubstitute for mocking
- Builder pattern for test data
- Descriptive names: Should_X_When_Y
```

Refactor prompt:

```
<!-- .github/prompts/refactor.prompt.md -->
---
mode: edit
---
Refactor #selection to improve readability
while maintaining behavior. Apply SOLID principles.
```

Custom Agents

Define domain experts in `.github/agents/`:

```
<!-- .github/agents/architect.agent.md -->
---
name: architect
description: Senior .NET architect for design decisions
tools: ['codebase', 'fetch']
---
You are a senior .NET architect at AllPhi.
When answering questions:
- Consider scalability and maintainability
- Reference our architecture documentation
- Suggest patterns appropriate for our team size
- Consider Azure deployment constraints
```

Invoke with: `@architect` in chat

GitHub Copilot Extensions

 **Important:** GitHub Copilot Extensions are deprecated (sunset November 2025). **MCP is the future-proof approach.**

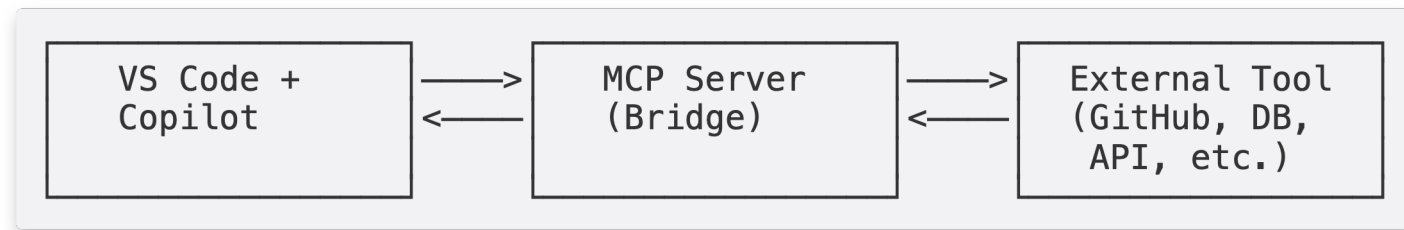
MCP Servers: The New Standard

Model Context Protocol (MCP) is an open standard by Anthropic that enables AI models to interact with external tools and data sources.

Why MCP?

- Open standard (backed by Google, Microsoft, OpenAI, Amazon)
- Works across AI tools (Copilot, Claude, Cursor, etc.)
- Same configuration works everywhere
- Growing ecosystem of servers

How MCP Works



MCP servers act as **bridges** between Copilot and external tools/data.

Two types:

- **Remote servers:** Hosted, no local setup (e.g., GitHub MCP Server)
- **Local servers:** Run on your machine, access local resources

Popular MCP Servers

Server	Purpose	Type
GitHub	Repos, issues, PRs, code search	Remote
Playwright	Browser automation, testing	Local
Azure	Azure resources and services	Remote
Postgres	Database queries and analysis	Local
Filesystem	Local file access	Local
Docker	Container management	Local

Discover more at the **GitHub MCP Registry**:
github.com/modelcontextprotocol/servers

Configuring MCP Servers in VS Code

Create `.vscode/mcp.json` in your workspace:

```
{
  "servers": {
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/v1",
      "headers": {
        "Authorization": "Bearer ${input:github-token}"
      }
    },
    "playwright": {
      "command": "npx",
      "args": ["@playwright/mcp-server"]
    }
  }
}
```

Or use the **MCP: Add Server** command in VS Code!

Using MCP Servers

1. **Enable Agent Mode** in Copilot Chat
2. Click the **Tools** icon () to see available MCP tools
3. Ask questions that leverage the tools:

```
Using the GitHub MCP server:  
"What are the open issues in my repo?"  
"Create a PR for this branch"  
"Search for authentication code across our org"
```

Note: Copilot asks permission before executing MCP tools. You can approve per-request or for the session.

MCP + Agent Mode = Power

Agent mode can **automatically use MCP tools** to complete complex tasks:

Example prompt:

```
Set up a new feature branch, implement the login endpoint, write tests, and create a draft PR
```

Agent with MCP:

1. Uses GitHub MCP to create branch
2. Writes code (standard Copilot)
3. Uses Playwright MCP to run E2E tests
4. Uses GitHub MCP to create PR

The combination unlocks **truly agentic workflows!**

Enterprise Features

Knowledge Bases (Enterprise tier):

- Index your documentation
- Access via `@github #kb`
- Being replaced by Copilot Spaces

Copilot Spaces (New):

- Organize code + docs together
- More flexible than Knowledge Bases
- Team collaboration features

Content Exclusions:

- Exclude sensitive paths from Copilot
- Repository, org, or enterprise level

Customization Strategy

Start simple, add as needed:

1.  Create `copilot-instructions.md` (Day 1)
2.  Add file-specific instructions (Week 1)
3.  Create prompt files for common tasks (Week 2-3)
4.  Build custom agents if needed (Month 2+)
5.  Explore extensions for your toolchain

Don't over-engineer: Most teams get 80% of value from instructions + prompt files alone.



Break

15 minutes

Part 3



Hands-on Lab

SupermarketReceipt Refactoring Kata

Angelo Dejaeghere

About the Kata

SupermarketReceipt-Refactoring-Kata by Emily Bache

A supermarket checkout system with:

- Products (rice, apples, toothbrushes)
- Pricing rules
- Special offers (% discount, multi-buy, bundles)

The challenge: The `handleOffers` method is "nasty code" - complex conditionals, poor readability, begging for refactoring.

Perfect for practicing with Copilot!

Lab Setup

```
# 1. Clone the repository
git clone https://github.com/emilybache/SupermarketReceipt-Refactoring-Kata.git

# 2. Navigate to TypeScript folder
cd SupermarketReceipt-Refactoring-Kata/typescript

# 3. Open in VS Code
code .

# 4. Install and verify
npm install
npm test
```

Verify Copilot is active: Look for Copilot icon in status bar ✓

Create Instructions File

Create `.github/copilot-instructions.md` :

```
# SupermarketReceipt Kata Guidelines

## Project Context
This is a refactoring exercise for a supermarket receipt system.
Focus on improving code quality without changing behavior.

## Tech Stack
- TypeScript 5
- Jest for testing

## Refactoring Goals
- Eliminate code smells
- Apply SOLID principles
- Maintain all existing tests passing
```

Exercise 1: Code Exploration (20 min)

Goal: Use Ask mode to understand the codebase

Tasks:

1. Open `src/ShoppingCart.ts` and find `handleOffers`
2. Ask Copilot: *"Explain this method and what discount types it handles"*
3. Ask: *"What code smells do you see in this method?"*
4. Ask: *"What refactoring would you suggest?"*

Tip: Select the method before asking for better context

Exercise 1: Expected Insights

Copilot should identify:

- ✗ Long method (too many responsibilities)
- ✗ Complex switch/if statements
- ✗ Magic numbers
- ✗ Mixed abstraction levels
- ✗ Feature envy

Suggested refactorings:

- Extract method per discount type
- Strategy pattern for discounts
- Replace conditionals with polymorphism

Exercise 2: Test Generation (30 min)

Goal: Use Copilot to write tests for discount scenarios

Tasks:

1. Open the test file
2. Use `/tests` or Ask mode to generate:
 - Test for 10% discount on 2kg apples
 - Test for "three for two" toothbrush offer
 - Test for "five for €X" pricing

Tip: Be specific about expected values!

Exercise 2: Sample Prompts

Generate a Jest test that verifies a 10% discount is correctly applied when buying 2kg of apples at €1.99/kg

Expected:

- Subtotal: €3.98
- Discount: €0.40 (rounded)
- Total: €3.58

Write a test for the "three for two" offer on toothbrushes priced at €0.99 each. Buying 3 should charge for 2.

Exercise 3: Refactoring (25 min)

Goal: Use Edit mode to safely refactor

Tasks:

1. Ensure all tests pass: `npm test`
2. Use Edit mode to extract discount calculation methods
3. Run tests after each change
4. Use Edit mode to introduce the Strategy pattern

Golden Rule: Tests must pass after every change!

Exercise 3: Refactoring Steps

Step 1: Extract percentage discount logic

Extract the percentage discount calculation into a separate private method called `calculatePercentageDiscount`

Step 2: Extract other discount types

Extract three-for-two and five-for-amount calculations into separate methods following the same pattern

Step 3: Introduce polymorphism (advanced)

Refactor the discount types to use the Strategy pattern with a `DiscountStrategy` interface

Bonus Challenge

Implement the "Bundle" feature:

“

"When you buy all items in a product bundle, you get 10% off the bundle"

”

Use Agent mode:

```
Implement a Bundle discount feature where buying all products  
in a bundle gives 10% off those items.  
Create tests first, then implement.
```

Wrap-up: Key Takeaways

1. **LLMs are powerful but imperfect** - Always review output
2. **Three modes for different tasks** - Ask → Edit → Agent
3. **Context is everything** - Better input = Better output
4. **Customize for your team** - Instructions, prompts, agents
5. **Practice makes perfect** - The kata is just the beginning!

Q&A

Questions?

Thank You

Happy Coding with Copilot! 🚀

Email: angelo@g3l.consulting

Website: <https://www.dev-craft.be/>

LinkedIn: <https://www.linkedin.com/in/angelodejaeghere/>